4 CSS Grid Properties (and One Value) for Most of Your Layout Needs

Anna Prenzel on Mar 30, 2020

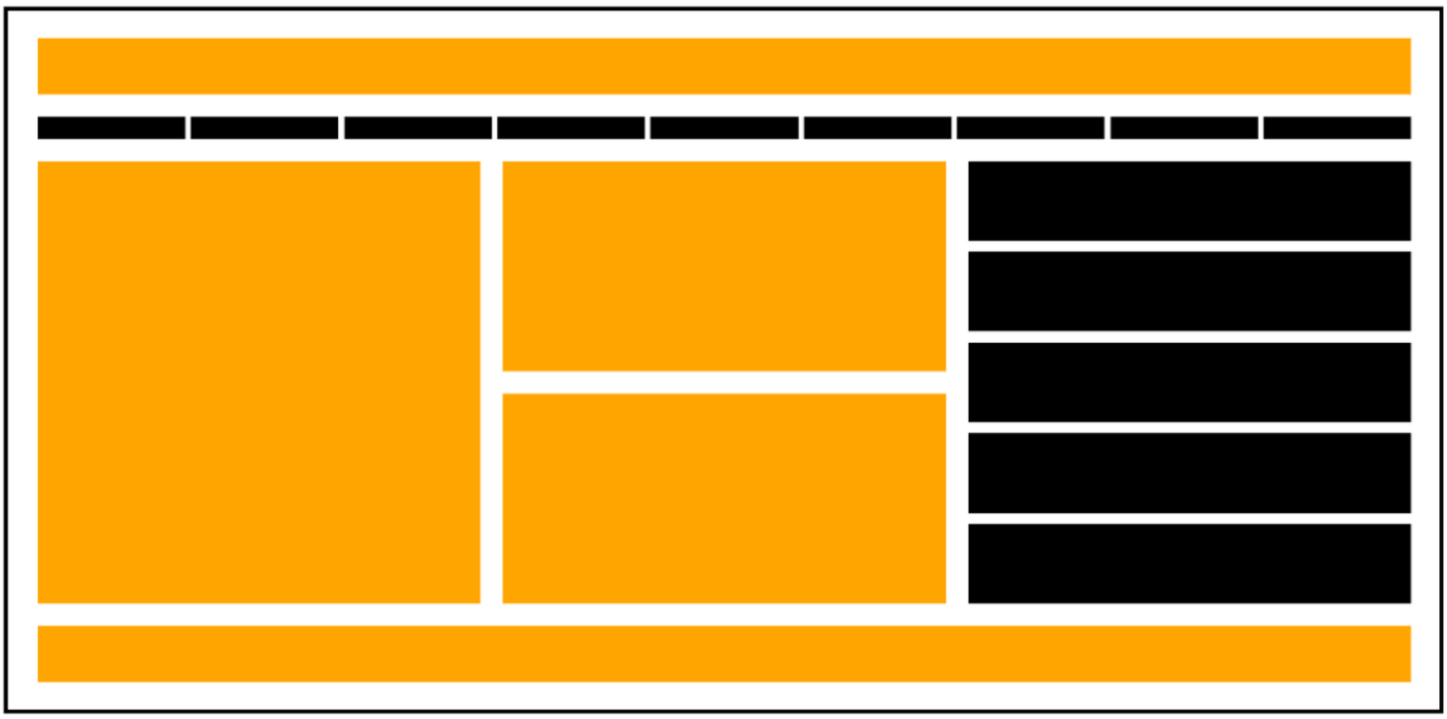
CSS Grid provides us with a powerful layout system for websites. The CSS-Tricks guide (https://css-tricks.com/snippets/css/complete-guide-grid/) gives you a comprehensive overview of Grid's properties with layout examples. What we're going to do here is a reverse approach to show you the smallest possible set of grid properties you need to know to meet most of your layout needs.

These five properties will get you up and running:

- `display` (for the `grid` value)
- `grid-template-columns`
- `grid-gap`
- `grid-auto-flow`
- `grid-column` / `grid-row`

Here's how simple it is. Let's assume you want to implement the following layout for small, medium and large screens.

Small and medium-sized screens



Large screen layout

This is the markup we'll be working with:

HTML

```
<!-- Stuff before -->

<nav class="container-nav">
  <ul>
    <li></li>
    <li></li>
    <li></li>
    <li></li>
    <li></li>
```

```
    <li></li>
    <li></li>
    <li></li>
    <li></li>
  </ul>
</nav>

<div class="container-main">
  <section class="item item-type-a"></section>
  <section class="item item-type-b"></section>
  <section class="item item-type-b"></section>
  <section class="item container-inner">
    <section class="item-inner"></section>
    <section class="item-inner"></section>
    <section class="item-inner"></section>
    <section class="item-inner"></section>
    <section class="item-inner"></section>
  </section>
</div>

<!-- Stuff after -->
```

If we apply a few baseline styles, this is what we get, which is already sufficient for small screens:

Embedded Pen Here

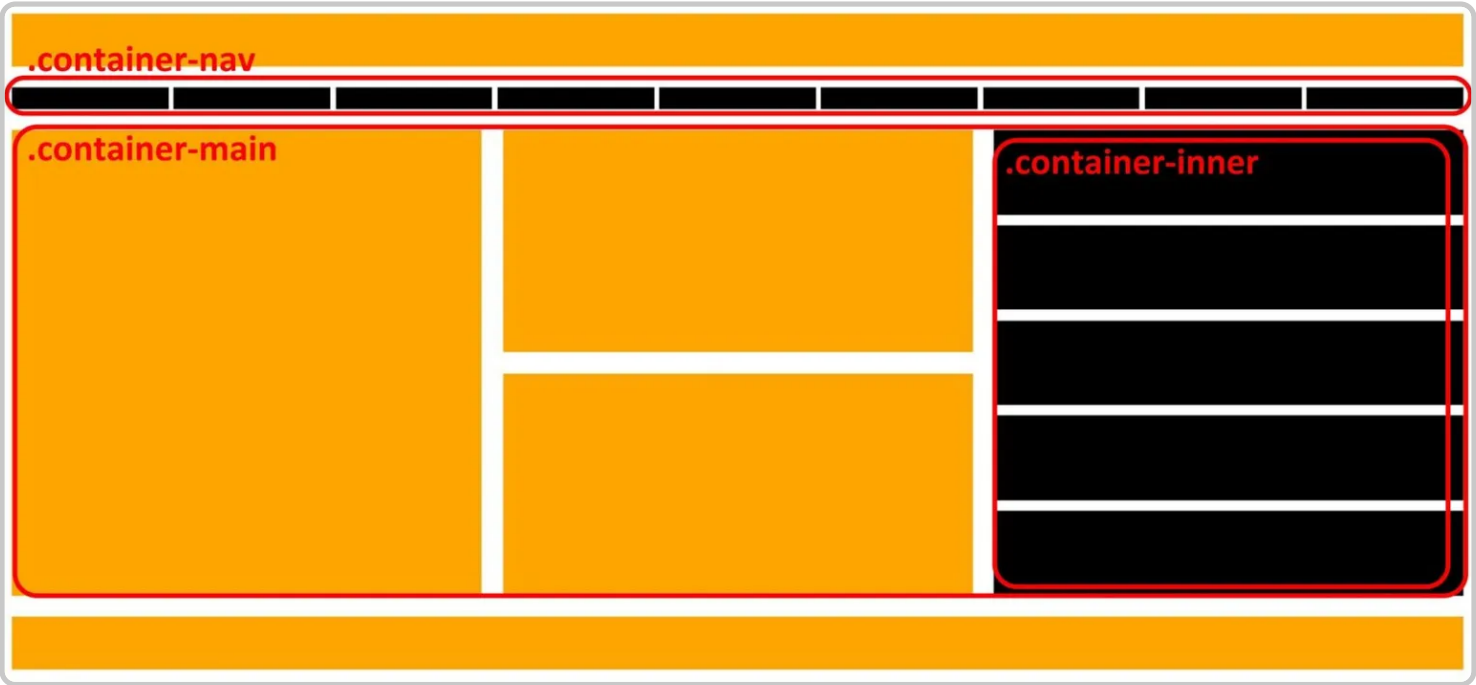
Now we can get into the grid properties!

(#aa-use-display-grid-to-divide-the-page-into-independent-layout-containers) Use display: grid to divide the page into independent layout containers

First, we need to determine which parts of the page should be aligned with grid layouts. It is possible to define a single grid layout for the whole page. However, for websites with a very

complex structure (e.g. news websites), handling a large grid quickly becomes complicated to wrangle. In this case, I recommend breaking things down into several, independent grid containers.

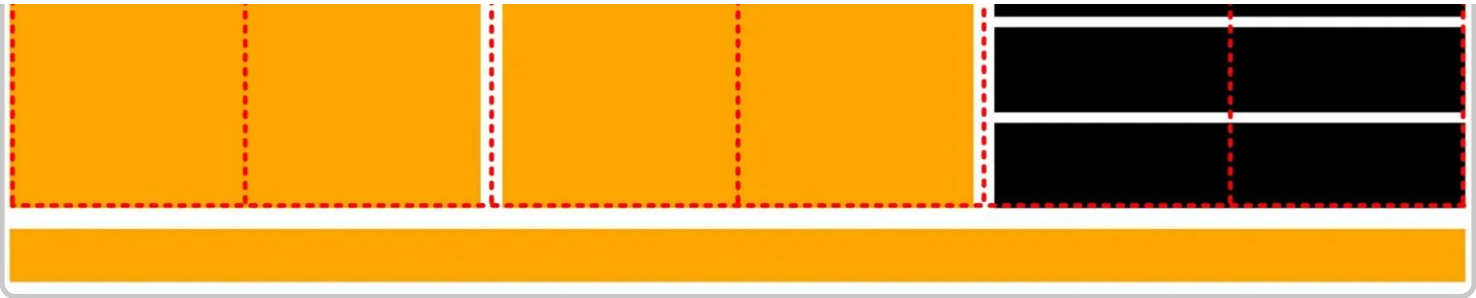
Like this:



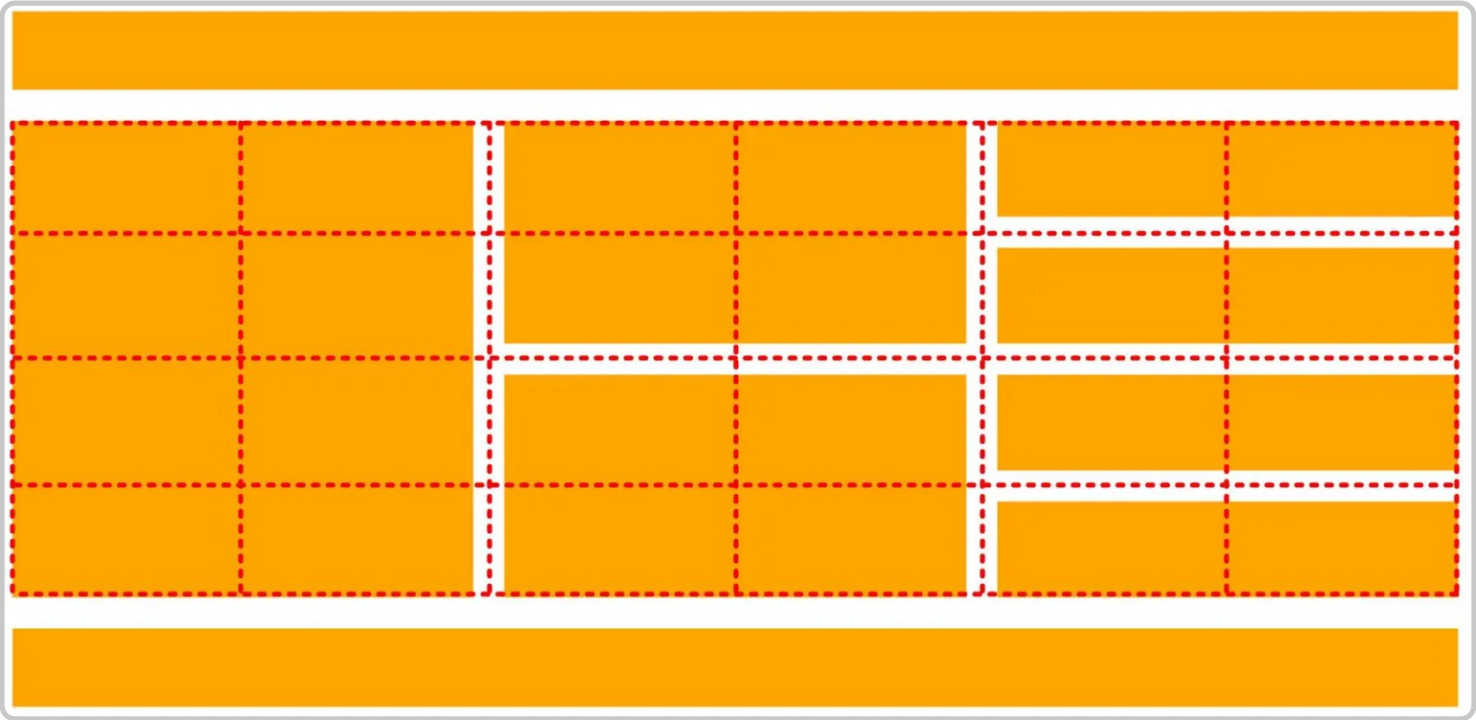
Where do you draw the line between what is and isn't a grid? Here's a personal rule of thumb I follow:

> *If the layout in a particular part of the page does not fit into the grid of an adjacent or surrounding part of the page, make that part its own grid container.*

I have drawn the grid lines into the page section with the class `.container-main` in the following image You may notice that the section with the `.container-inner` class from the markup does not fit exactly into the grid of rows.



Here's another possible layout where the small sections fit into the surrounding grid if a finer line raster is chosen. A separate grid container is not absolutely necessary here.



To kick this off, let's `.container-main` into a grid container. This is the basic building block for CSS Grid — turning an element into a grid container with the `display` property:

```
.container-main {
  display: grid;
}
```

We'll want to do the same with our other grid containers:

```
.container-inner {
  display: grid;
}

.container-nav {
  display: grid;
```

```
}
```

(#aa-use-grid-template-columns-to-define-the-required-columns) Use grid-template-columns to define the required columns

Next, we're going to define the number of columns we need in each grid container and how wide those columns should be. My guideline for the number of columns: **use the smallest common multiple of the maximum number of columns required for the different screen sizes.**

How does that work? The `.container-main` element has a total of two columns on medium-sized screens. If we take that and multiply it by the number of columns on large screens (three), we get a total of six columns.

We can do the same for our navigation, the `.container-inner` element. There are three columns on medium-sized screens, which we multiple by one column on large screens to get a total of three columns.

The `.container-nav` element provides no number of columns. In this case, the grid system should automatically adjust the number of columns to the number of menu elements. It's common to add or remove items in a navigation, and it'd be great if it responded accordingly, which is something grid can help us with a little later on.

OK, so we defined the number of columns for each grid container. Let's use the `grid-template-columns` property to set those into place. But, first a couple of minor details:

- The `grid-template-columns` property is only used on the grid container. In other words, you won't find it being used (at least correctly) on a grid item inside the container.
- The property accepts a bunch of different values that both define the number of columns and how wide they should be. The one we're interested in here is the fractional (`fr`) unit. I'd highly suggest checking out Robin's overview (https://css-tricks.com/introduction-fr-css-unit/)

because it's unique to grid and does an amazing job doing calculations to decide how grid elements fit inside a grid container.

We need six equal-width columns in `.container-main`. We can write that like this:

```
.container-main {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr 1fr 1fr 1fr;
}
```

Or, we can turn to the `repeat()` function to simplify it into something more readable:

```
.container-main {
  display: grid;
  grid-template-columns: repeat(6, 1fr);
}
```

Let's take that knowledge and apply it to our `.container-inner` element as well, which we decided needs three columns.

```
.container-inner {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

(#aa-use-grid-gap-to-add-spacing-between-grid-items) Use grid-gap to add spacing between grid items

By default, grid uses all the space it has in a grid container to fit in grid items. Having elements flush next to one another might be a design requirement, but not for the particular layout we're making. We want some breathing room between things!

We have the `grid-gap` property for that. Again, this is a property that's just for grid containers and what it does is create vertical and horizontal spacing between grid items. It's actually a

shorthand property that combines the vertical spacing powers of `grid-row-gap` and horizontal spacing powers of `grid-column-gap`. It's handy that we're able to break things out like that but, in times like this where we're working with the same amount of spacing in each direction, the shorthand `grid-gap` is much nicer to write.

We want `20px` of space between grid items in `.container-main`, `10px` of space in `.container-inner`, and `5px` of space in `.container-nav`. No problem! All it takes is a one-liner on each grid container.

```
.container-main{
  display: grid;
  grid-template-columns: repeat(6, 1fr);
  grid-gap: 20px;
}

.container-inner {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-gap: 10px;
}

.container-nav {
  display: grid;
  grid-gap: 5px;
}
```

(#aa-use-grid-column-and-grid-row-to-determine-the-size-of-the-individual-grid-items) Use grid-column and grid-row to determine the size of the individual grid items

Now it is time to put the layout into the shape we want it!

First is the `grid-column` property, which allows us to extend a grid item across n columns, where n is the number of columns to span. If you're thinking this sounds an awful lot like the `rowspan` attribute that lets us extend cells across multiple rows in HTML tables, you wouldn't be wrong.

It looks like this when we use it on a grid `.item` in our `.container-main` element, and on the `.inner-item` elements in `.container-inner`:

```
.item {
  grid-column: span 6;
}

.item-inner {
  grid-column: span 3;
}
```

What we're saying here is that each item span six rows in our main container and three rows in our inner container — which is the total number of columns in each container.

Hey! An interesting thing about CSS Grid is that we are able to name the lines of the grid (https://css-tricks.com/things-ive-learned-css-grid-layout/#article-header-id-4) . They come with implicit names (https://css-tricks.com/difference-explicit-implicit-grids/) out of the box but naming them is a powerful way to distinguish between the starting and ending lines for each column on the track.

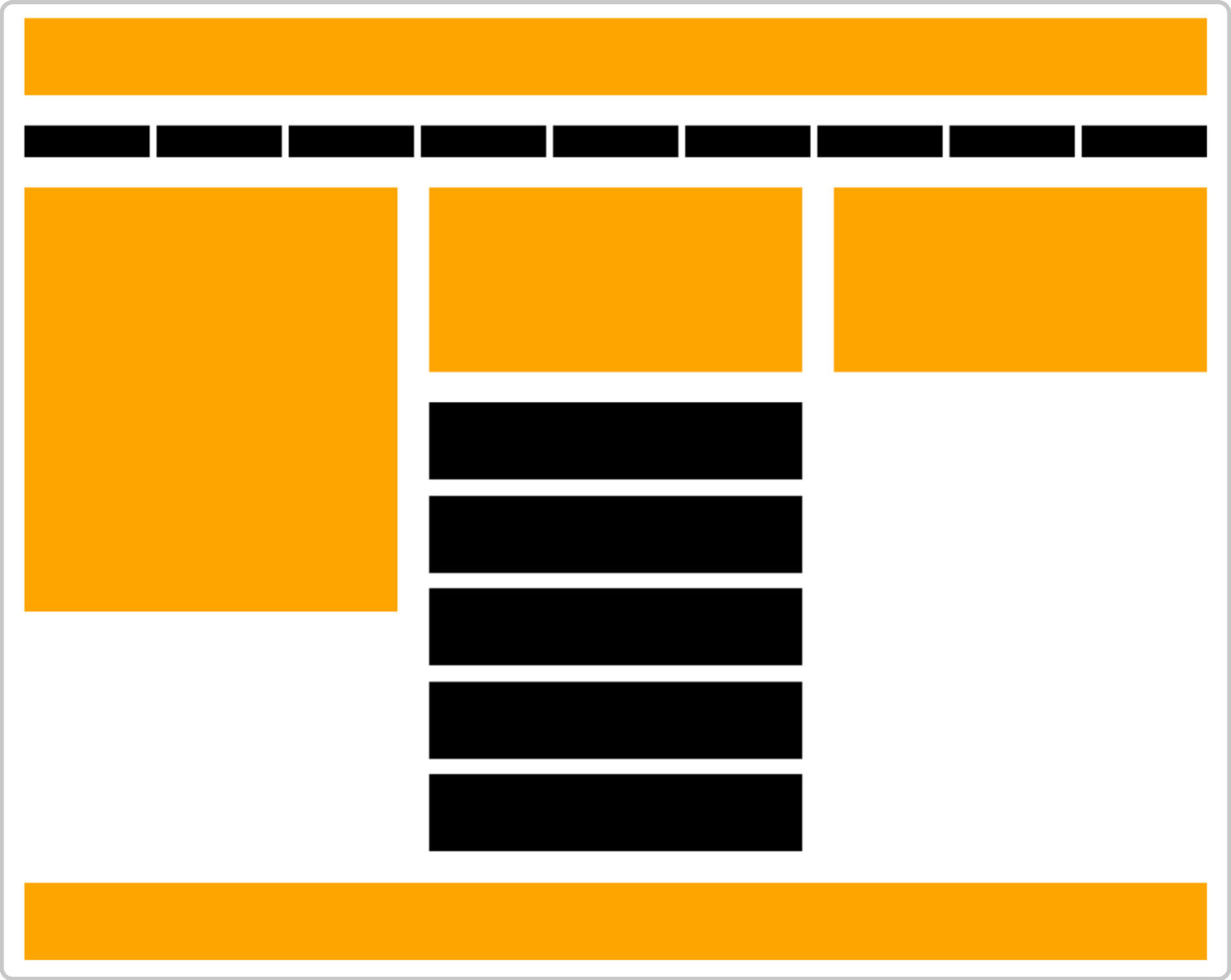
We can change the number of columns and rows the items should span at different breakpoints:

```
@media screen and (min-width: 600px) {
  .item-type-b {
    grid-column: span 3;
  }

  .item-inner {
    grid-column: span 1;
  }
}

@media screen and (min-width: 900px) {
  .item {
    grid-column: span 2;
    grid-row: span 2;
  }

  .item-type-b{
    grid-row: span 1;
  }

  .item-inner{
    grid-column: span 3;
```

```
  }
}
```

(#aa-using-grid-auto-flow-to-control-the-placing-of-the-elements) Using grid-auto-flow to control the placing of the elements

CSS Grid places elements one row after the other. This is why the result in our example looks like this at the moment:



A column-by-column placement can be achieved by setting the `grid-auto-flow` property to

column (`row` is the default value). Our layout will profit from column-wise placement in two cases. First, it makes our menu items finally appear in a horizontal orientation. Secondly, it brings the elements of the container class into the desired grouping.

(#aa-the-final-result) The final result

Embedded Pen Here

(#aa-conclusion-more-or-less-specification) Conclusion: More or less specification?

The grid system allows us to work under the motto, “make as many specifications as necessary, but as few as possible.” We’ve only covered a few of the specifications necessary to turn elements into a CSS grid container and the items inside it into grid items for the sake of showing just how little you need to know to build even complex layouts with CSS Grid.

CSS Grid supports additional use cases where:

- We want to make even fewer specifications in order to instead rely more on automatic positioning.
- We want to make even more specifications in order to determine more details of the resulting layout.

If the first case applies, then it’s worth considering the following additional grid options:

- When creating the grid with `grid-template-columns`, you can have the grid system

automatically determine the width of individual columns with the auto keyword or adapt it to the existing content with the settings `min-content`, `max-content`, or `fit-content`.

- You can let the grid system automatically determine the number of required columns with the help of `repeat`, `auto-fill`, `auto-fit`, and `minmax`. Even media queries can become redundant and these tools help make things flexible without adding more media queries.

Here are a couple of articles on the topic that I recommend: Becoming a CSS Grid Ninja! (https://medium.com/@elad/becoming-a-css-grid-ninja-f4c6db018cc1) and Auto-Sizing Columns in CSS Grid: auto-fill vs. auto-fit (https://css-tricks.com/auto-sizing-columns-css-grid-auto-fill-vs-auto-fit/) .

If the second case applies, CSS Grid offers even more settings options for you:

- You can explicitly specify the width of the columns in the unit of your choice (e.g. `px` or `%`) using the `grid-template-columns` property. In addition, the property `grid-template-rows` is available to define the number and width of rows, should there be a specific number of them.
- You can also define specific column or row numbers for positioning as values for `grid-column` and `grid-row` (or use the properties `grid-column-start`, `grid-column-end`, `grid-row-start`, or `grid-row-end`).

And we haven't even gotten into CSS Grid alignment (https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout/Box_Alignment_in_CSS_Grid_Layout) ! Still, the fact that we can accomplish so much without even broaching that topic shows how powerful CSS Grid is.